

FlyRank Week 2 Assignment - The Prompt Ladder

FlyRank AI Internship

Week 2 Assignment - The Prompt Ladder

Name: Achyuta Biswal **Track:** Backend AI Engineering

Objective

This assignment demonstrates how a single weak prompt can be incrementally improved into a production-grade prompt by adding **one distinct improvement layer at a time**. Each rung of the ladder isolates a single variable — goal clarity, context, output format, constraints, or quality criteria — so that the specific effect of that layer on the AI’s output can be observed and evaluated honestly.

The ladder consists of **six prompt runs**: a weak baseline followed by five incremental versions.

Ladder Overview

Version	Layer Added	Focus
Baseline	—	Weak, vague prompt
Version 1	Clear Goal	Defines <i>what</i> to build
Version 2	Real Context	Defines <i>the environment/stack</i>
Version 3	Specified Output Format	Defines <i>how the answer should be structured</i>
Version 4	Constraints	Defines <i>boundaries and non-negotiables</i>
Version 5	Quality Criteria	Defines <i>what “good” looks like</i>

Baseline (Weak Prompt)

Write some Java backend code.

Representative AI Output (Excerpt)

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, this is a backend example.");
        // TODO: add database connection
        // TODO: add API logic
    }
}
```

Notes

- What changed in the prompt:** Nothing — this is the unmodified starting prompt with no goal, context, or format.
- What actually improved in the output:** Not applicable; this is the baseline against which later versions are measured.
- What still failed:** The output is not backend code in any meaningful sense — it is a `main` method with placeholder comments and no framework, no API, no persistence layer, and no indication of what “backend” even means here.
- What I would try next:** State an explicit, unambiguous goal describing what kind of backend system should be built.

Version 1 — Clear Goal

Create a Java backend application that exposes a REST API for managing a list of products.

Representative AI Output (Excerpt)

```
@RestController
public class ProductController {

    private List<String> products = new ArrayList<>();

    @GetMapping("/products")
    public List<String> getProducts() {
        return products;
    }

    @PostMapping("/products")
    public void addProduct(@RequestBody String product) {
        products.add(product);
    }
}
```

Notes

1. **What changed in the prompt:** Added a single, explicit goal — “REST API for managing a list of products” — replacing the vague “some Java backend code.”
2. **What actually improved in the output:** The response produced an actual `@RestController` with real HTTP endpoints (GET / POST) instead of a placeholder `main` method, and the domain (products) was correctly reflected in the code.
3. **What still failed:** The AI assumed Spring Boot on its own since no framework or stack was specified, products are stored as raw `String` values in an in-memory `ArrayList` with no entity, database, or persistence layer, and there is no indication of Maven, Java version, or database technology.
4. **What I would try next:** Provide the real technology context (framework, build tool, Java version, database) so the AI stops guessing the environment.

Version 2 — Real Context

Create a Java backend application that exposes a REST API for managing a list of products.
Use Java 23, Spring Boot 3, Maven as the build tool, and MySQL as the database with Spring Data JPA for persistence.

Representative AI Output (Excerpt)

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private double price;
    // getters and setters
}

public interface ProductRepository extends JpaRepository<Product, Long> {
}

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository repository;

    @GetMapping
    public List<Product> getAll() {
        return repository.findAll();
    }
}
```

Notes

1. **What changed in the prompt:** Added real technology context — Java 23, Spring Boot 3, Maven, MySQL, and Spring Data JPA — without changing the underlying goal.
2. **What actually improved in the output:** The response now used a proper `@Entity` class backed by an actual database table, replaced the raw `ArrayList` with a `JpaRepository` interface for persistence, and used constructor-style dependency injection patterns consistent with Spring Boot 3.
3. **What still failed:** The entity was exposed directly through the controller instead of using a DTO, field injection (`@Autowired` on a field) was used instead of constructor injection, which is considered poor practice, and there was no indication of how the response should be organized (no headings, no structure, just a code dump).
4. **What I would try next:** Specify the exact output structure so the answer is organized into clearly separated, easy-to-review sections rather than a single unstructured block of code.

Version 3 — Specified Output Format

Create a Java backend application that exposes a REST API for managing a list of products. Use Java 23, Spring Boot 3, Maven as the build tool, and MySQL as the database with Spring Data JPA for persistence.

Structure your response using the following sections, in this order:

1. Project Structure
2. Entity
3. Repository
4. Service
5. Controller
6. application.properties

Representative AI Output (Excerpt)

```
## 1. Project Structure
src/main/java/com/example/product/
├── entity/
├── repository/
├── service/
└── controller/

## 2. Entity
@Entity
public class Product { ... }

## 3. Repository
public interface ProductRepository extends JpaRepository<Product, Long> { }

## 4. Service
@Service
public class ProductService { ... }

## 5. Controller
@RestController
@RequestMapping("/products")
public class ProductController { ... }

## 6. application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/productdb
spring.jpa.hibernate.ddl-auto=update
```

Notes

1. **What changed in the prompt:** Added an explicit, numbered output structure that the response must follow, without altering the goal or the technology context.
2. **What actually improved in the output:** The response was organized into clearly labeled, navigable sections matching the requested order, and a **Service layer was introduced for the first time**, separating business logic from the controller instead of calling the repository directly from it.
3. **What still failed:** The response still exposed the raw entity through the API instead of introducing a DTO layer, field-level validation was completely absent (a product could be saved with a blank name or negative price), and no constraints were given on code style, error handling, or what NOT to do.
4. **What I would try next:** Add explicit constraints — including a required DTO layer, validation rules, and architectural boundaries — so the AI stops making silent assumptions about correctness.

Version 4 — Constraints ⚠ (Honest Regression Included)

Create a Java backend application that exposes a REST API for managing a list of products.
Use Java 23, Spring Boot 3, Maven as the build tool, and MySQL as the database with Spring Data JPA for persistence.

Structure your response using the following sections, in this order:

1. Project Structure
2. Entity
3. Repository
4. Service
5. Controller
6. application.properties

Constraints:

- Do NOT expose the entity directly in the controller; use DTOs for all request/response bodies.
- Apply Bean Validation annotations on all DTO fields.
- Do NOT use field injection; use constructor injection only.
- Do NOT include business logic in the controller layer.

Representative AI Output (Excerpt)

```
public class ProductRequestDTO {
    @NotBlank
    private String name;

    @Positive
    private double price;
    // getters and setters
}

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<ProductResponseDTO> create(@Valid @RequestBody ProductRequestDTO dto) {
        return ResponseEntity.ok(service.create(dto));
    }
}
```

Notes

1. **What changed in the prompt:** Added four explicit constraints (DTO-only exposure, Bean Validation, constructor injection, and no business logic in the controller), with no change to the goal, context, or format.
2. **What actually improved in the output:** The response introduced a proper `ProductRequestDTO` with `@NotBlank` and `@Positive` validation annotations, and the controller switched from field injection to clean constructor injection with a `final` service field.
3. **What still failed (honest regression):** Because the constraints list grew long and specific, the AI became overly focused on **satisfying each individual constraint as a checklist** rather than reasoning about the design holistically — it started adding excessive boilerplate (e.g., a duplicate DTO for every possible action, redundant mapper methods) that increased verbosity without improving correctness. This is a genuine case where adding a layer made the output *bulkier and harder to read*, not simply better. The root cause was that constraints alone tell the AI what to avoid but not what “good” looks like holistically, so it over-corrected defensively.
4. **What I would try next:** Add explicit quality criteria (e.g., “favor clean, minimal, idiomatic code” and “follow SOLID principles”) so the AI is guided toward good *design judgment* rather than mechanically checking off constraints.

Version 5 — Quality Criteria

Create a Java backend application that exposes a REST API for managing a list of products. Use Java 23, Spring Boot 3, Maven as the build tool, and MySQL as the database with Spring Data JPA for persistence.

Structure your response using the following sections, in this order:

1. Project Structure
2. Entity
3. Repository
4. Service
5. Controller
6. application.properties

Constraints:

- Do NOT expose the entity directly in the controller; use DTOs for all request/response bodies.
- Apply Bean Validation annotations on all DTO fields.
- Do NOT use field injection; use constructor injection only.
- Do NOT include business logic in the controller layer.

Quality Criteria:

- Follow SOLID principles and standard Java naming conventions.
- Keep the code minimal, idiomatic, and free of redundant classes or boilerplate.
- Follow REST best practices (proper HTTP status codes, proper verb usage, meaningful URIs).
- Prioritize readability and maintainability over exhaustive coverage of edge cases.

Representative AI Output (Excerpt)

```
@Service
public class ProductService {

    private final ProductRepository repository;
    private final ProductMapper mapper;

    public ProductService(ProductRepository repository, ProductMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    public ProductResponseDTO create(ProductRequestDTO dto) {
        Product saved = repository.save(mapper.toEntity(dto));
        return mapper.toDto(saved);
    }
}
```

```
@PostMapping
public ResponseEntity<ProductResponseDTO> create(@Valid @RequestBody ProductRequestDTO dto) {
    ProductResponseDTO created = service.create(dto);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}
```

Notes

1. **What changed in the prompt:** Added explicit quality criteria (SOLID, minimalism, REST best practices, readability priority) on top of the existing constraints, format, context, and goal.
2. **What actually improved in the output:** The controller now correctly returned `HttpStatus.CREATED` (201) for a resource-creation endpoint instead of a generic 200 OK, a single dedicated `ProductMapper` class replaced the earlier redundant duplicate-DTO pattern from Version 4, and the service layer showed clear single-responsibility separation between mapping and persistence logic.
3. **What still failed:** The response still did not include centralized exception handling (e.g., a `@ControllerAdvice` for validation errors), and pagination or filtering for the `GET /products` endpoint was still absent despite being a common real-world REST requirement.
4. **What I would try next:** Add a follow-up refinement round explicitly requesting centralized exception handling and pagination, treating them as a natural sixth layer for a future iteration beyond this assignment's scope.

Final Reusable Prompt

The following prompt consolidates all five layers into a single, copy-paste-ready prompt. Any Backend AI Engineering intern can use it without modification to generate a well-structured Spring Boot REST API starting point.

Generate a Java Spring Boot REST API project for managing products, using the following technology stack:

- Java 23
- Spring Boot 3
- Maven
- MySQL
- Spring Data JPA

Apply the following architectural and design requirements:

- Layered Architecture (Controller → Service → Repository)
- DTOs for all request/response bodies (never expose entities directly)
- Bean Validation on all DTO fields
- SOLID Principles
- REST best practices (correct HTTP status codes, correct verb usage, meaningful resource-based URIs)
- Constructor-based dependency injection only (no field injection)
- No business logic inside controllers

Structure your response using exactly the following sections, in this order:

1. Project Structure
2. Dependencies
3. Entity
4. DTO
5. Repository
6. Service
7. Controller
8. application.properties
9. API Endpoints
10. Best Practices
11. Future Improvements

Keep the code minimal, idiomatic, and readable. Avoid redundant classes or unnecessary boilerplate. Prioritize clarity and maintainability over exhaustive edge-case handling.

Reflection

This exercise reshaped how I think about prompting an AI for engineering tasks. The most important lesson was that **each additional layer must be evaluated on its own**, because improvements do not stack neatly — Version 4 proved that adding constraints without accompanying quality guidance can actually make output worse by encouraging mechanical, checklist-driven bloat rather than good design judgment. I also learned that a “goal” and “context” are not the same thing: stating what to build is meaningless without also stating the environment it must live in, and neither matters if the output format is left unspecified, since unstructured answers are hard to evaluate or reuse. Quality criteria turned out to be the layer that gave the AI the most creative latitude to self-correct, since it addressed *judgment* rather than *rules*. Overall, this ladder taught me that strong prompting is less about piling on requirements and more about sequencing them so each layer resolves a specific ambiguity the AI would otherwise have to guess at — and that honestly documenting failures, not just wins, is what makes a prompt-engineering process genuinely improvable.

End of Week 2 Assignment.